Chapter 11 worked solutions • File-system interface

Answers use the same IDs as the exercises and retain their premises, derivations, and counterexamples.

OS11-E01 • Symlinks followed by mount crossings

1. Resolve root→a, read link as ../b, and interpret that relative target from a. The .. reaches root, b crosses to F7's root, and the suffix is empty. The answer is F7 root, without an invented filename.
2. Reading other resets to root with pending x, y,.., z. Resolve /x/y, then .. to /x and z to /x/z. Canceling other/.. first would incorrectly yield /a/z. Every intermediate directory still requires search permission.
3. A hard link adds a name for the same inode within one filesystem. A symlink is a separate pathname object, can cross filesystems, and may dangle. Names can change between check(path) and open(path). A trusted dirfd fixes only the starting point, and final O_NOFOLLOW does not constrain intermediate components. Use a protected reference or appropriately restricted whole-path resolution. A mount point is not a symlink.

OS11-E02 • Actual OFD offsets

1. Both start at zero. P receives ABC and O1 becomes 3. C.fd4 also refers to O1, so C receives DE and O1 becomes 5. Q receives ABCD and O2 becomes 4. The final pair is (5,4).
2. Pread reads BC from byte 1 without changing O1=5. Read(fd4,2) then returns FG, making O1=7 while O2 stays 4. Inode sharing does not merge independent descriptions.
3. Another caller can change O1.offset between lseek and read. Pread puts position into one call and leaves the shared cursor unchanged. It does not stop another writer changing the contents, create a snapshot, or establish durability. Record the actual OFD at each step, not merely parent versus child.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 11 worked solutions • File-system interface (continued)

OS11-E03 • Named users and the mask

1. Alice receives owner rw without the mask. Bob selects named-user w, but w intersect r is empty. He cannot fall back to dev's r. Other dev members receive group r constrained by mask r.
2. Bob obtains w, still without r; dev rights are not unioned into his named entry. Displayed group-class mode bits represent the mask and change from r to rw. They do not remain a purported unchanged 0640.
3. An ordinary local Linux regular-file descriptor retains its previously authorized writable access mode, so its writes can still succeed. A new O_WRONLY open fails under the new mask. Immediate recall of existing authority requires another revocation mechanism. Windows deny-entry composition is not the rule for this POSIX ACL.

OS11-E04 • Record positions and index I/O

1. $77 \cdot 32 = 2464 = 2 \cdot 1024 + 416$. Total data is 32000 B, versus 32768 B of allocated capacity, giving 768 B waste. The last block holds eight records, or 256 B. Do not charge 768 B to every block.
2. The index is $32 \cdot 16 = 512$ B. Its greatest first key at most 77 is 64, selecting block 2 with keys 64…95. Binary search needs at most $\lfloor \log_2 32 \rfloor + 1 = 6$ comparisons plus one data-block I/O. An uncached index adds its actual layout-dependent reads.
3. jL assumes every preceding record has length L. Variable-length positioning requires summing prior lengths or storing an offset index. Record number and average length do not determine an exact byte address. A file-relative block is not a device LBA.

Chapter 11 worked solutions • File-system interface (continued)

OS11-E05 • Short writes and zero progress

1. Buffer starts are 0,2500,4000; requests are 6000,3500,2000. Done becomes 2500,4000,6000 and offsets 2600,4100,6100. Advance by the returned count, not the requested count.
2. Maintain $0 \leq done \leq N$, with [0, done) delivered in order and only the remainder pending. A positive k preserves that prefix and decreases the remainder by k. Positive progress each time implies completion within N successes. Errors or permanent lack of readiness do not promise successful termination.
3. Zero from a positive-count write is no progress; report or handle it under the API instead of spinning. Retry EINTR when no bytes transferred. On EAGAIN, wait for readiness. A positive-count read returning zero commonly indicates EOF; that is different from write progress. Failure behavior belongs in the contract.

OS11-E06 • An unnamed but live object

1. Link count starts at 2, becomes 1 after unlink a, then 0 after unlink b. O1 still retains the object, so it cannot be reclaimed. Truncate creates no name and closes no reference. Only the last close with no other references permits reclamation. Real mmap references also matter; the question excludes them.
2. Size becomes zero but O1.offset remains 7. A positive-count read beyond EOF returns zero; truncate does not seek back to zero.
3. No. Rename atomically changes the name mapping. New lookups reach the new inode, while an old fd retains the old object. An unnamed old object still waits for its final reference. Keep link counts and open references separate.

OS11-E07 • Mount hiding and permissions

1. A new lookup of old fails with ENOENT in F7; new reaches F7:new; P retains F0:old. F0 data is hidden from new lookup without being copied or deleted.
2. Other has directory --x, allowing search of known names but not enumeration. File other=r-- permits a read-open through the valid known path.
3. Yes. Parent wx permits search and removal of an entry. File mode000 is not the ordinary unlink authorization criterion. Other path permissions still apply, while sticky and additional restrictions are explicitly excluded. Directory w and file w govern different objects.

Chapter 11 worked solutions • File-system interface (continued)

OS11-E08 • Two groups and a combined request

1. Read passes through dev r; write through qa w. One rw request fails because no matching group entry includes the entire request. Arbitrarily unioning entries changes the rule.
2. Read still passes, while write and rw fail because the mask lacks w. Matched but insufficient groups do not fall back to other.
3. Matching named-user --- denies without group/other fallback. This is class-selection precedence, not generalized deny-entry synthesis. Test the complete current requested set against at least one complete matching group entry.

OS11-E09 • Visibility, exclusion, and persistence

1. Immediate-image values are B=1, B=1, C=1. Session snapshots give B=0, B=0, C=1, assuming A publishes on close and no other writer intervenes. Real network filesystems are not automatically one of these abstractions.
2. Advisory locks cannot stop a caller ignoring them. The proof assumes every conflict obtains a compatible lock. The half-open ranges do not overlap because their maximum start and minimum end are both 100. Locking alone also establishes neither cache validation nor flushing.
3. Within the assumed same-filesystem/same-directory model, complete tmp data and metadata with short-write handling, successfully fsync tmp, rename over target, then successfully fsync the parent directory before promising a durable name. Handle every error. Atomic name visibility is separate from directory persistence and does not create a cross-filesystem transaction. Establish visibility, authority, exclusion, and durability as separate contracts.

Chapter 11 worked solutions • File-system interface (continued)

Verification code appendix: file_interface_models.py

```
"""Self-contained deterministic file-interface models; no host filesystem mutation."""
from dataclasses import dataclass
from bisect import bisect_right

@dataclass
class OFD:
    data: bytes
    offset: int = 0
    def read(self,count):
        if count<0:raise ValueError('negative count')
        out=self.data[self.offset:self.offset+count];self.offset+=len(out)
        return out
    def pread(self,count,offset):
        if count<0 or offset<0:raise ValueError('negative argument')
        return self.data[offset:offset+count]

def offset_trace():
    o1,o2=OFD(b'ABCDEFGHIJ'),OFD(b'ABCDEFGHIJ')
    p={3:o1,4:o1};child=p.copy();q={8:o2};trace=[]
    for name,handle,count in [('P3',p[3],3),('P4',p[4],2),('Q8',q[8],2),('C3',child[3],2)]:
        data=handle.read(count);trace.append((name,data.decode(),o1.offset,o2.offset))
    trace.append(('pread',p[3].pread(2,1).decode(),o1.offset,o2.offset))
    return trace

def
acl_allows(uid,groups,req,*,owner='alice',filegroup='dev',owner_perm=6,users=None,group_perm=
4,named_groups=None,mask=6,other=4):
    """POSIX-style ACL requested-set rule, no privilege or LSM overrides."""
    users={} if users is None else users
    named_groups={} if named_groups is None else named_groups
    for v in [req,owner_perm,group_perm,mask,other,*users.values(),*named_groups.values()]:
        if not isinstance(v,int) or not 0<=v<=7:raise ValueError('permission bits')
    has=lambda perm:perm&req==req
    if uid==owner:return has(owner_perm)
    if uid in users:return has(users[uid]) and has(mask)
    matches=[]
    if filegroup in groups:matches.append(group_perm)
    matches.extend(perm for group,perm in named_groups.items() if group in groups)
    if matches:return has(mask) and any(has(perm) for perm in matches)
    return has(other)

def record_location(record,length=32,block=1024):
    if record<0 or length<=0 or block<=0:raise ValueError('invalid layout')
```

Chapter 11 worked solutions • File-system interface (continued)

```
    byte=record*length;return byte,byte//block,byte%block

def index_candidate(first_keys,key):
    if not first_keys or any(a>=b for a,b in zip(first_keys,first_keys[1:])):raise
ValueError('strict sorted index')
    pos=bisect_right(first_keys,key)-1;return pos if pos>=0 else None

def partial_write(total,initial,completions):
    if total<0 or initial<0:raise ValueError('negative size/offset')
    done=0;rows=[]
    for count in completions:
        if done==total:break
        if not 0<count<=total-done:raise ValueError('no progress or invalid completion')
        requested=total-done;start=done;done+=count
        rows.append((start,requested,count,done,initial+done))
    return rows,done==total

def overlap(a,b):
    if a[0]<0 or a[0]>=a[1] or b[0]<0 or b[0]>=b[1]:raise ValueError('empty/reversed range')
    return max(a[0],b[0])<min(a[1],b[1])

def reachable(edges,roots):
    seen=set();todo=list(roots)
    while todo:
        node=todo.pop()
        if node in seen:continue
        seen.add(node);todo.extend(edges.get(node,()))
    return seen

def directory_cycle_example():
    edges={'A':['B'],'B':['A']}
    return {v:sum(v in dest for dest in edges.values()) for v in edges},reachable(edges,[])

if __name__=='__main__':
    for row in offset_trace():print(row)
    print('record77',record_location(77))
    print('partial write',partial_write(6000,100,[2500,1500,2000]))
    for who,groups in [('alice',set()),('bob',{'dev'}),('carol',{'dev','qa'})]:
        print(who,{r:acl_allows(who,groups,r,users={'bob':2},named_groups={'qa':2}) for r in
[4,2,6]})
    print('cycle',directory_cycle_example())
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study